

AI-Based Food Recognition with Nutrition-Aware Recommendations

Jaspreet Singh*, Inderpal Singh†

*† Department of Computer Science and Engineering

National Institute of Technology Srinagar

Kashmir 190006, India

Email: [student email]

Project Guide: Dr. Lavanya Madhuri Bollipo

Project Co-Guide: Dr. Insha Majeed

Abstract—This article describes an educational Indian-food recognition application and a reproducibility audit of its evidence. The application joins YOLO11s detection, database nutrition per 100 g, user-estimated gram portions, an explicitly saved browser-only meal journal, and context-labeled recommendations. The existing export contains 48,693 images across 72 classes. A full read-only audit found 138 exact-duplicate groups spanning the original partitions, affecting 276 images, and strict annotation issues in 65 images. Consequently, the previously reported test mAP@50 of 0.820 and mAP@50:95 of 0.680 are historical observations, not estimates of clean independent generalization. Nutrition lookup is available for 19/72 classes with normalized exact matching, 55/72 with automatic matching without precomputed mappings, and 72/72 with the current mapping layer; these counts do not measure nutritional correctness. The work supplies fingerprinted audit artifacts, a source-family-grouped candidate split with quarantine, and an independent mapping-review protocol. It demonstrates an inspectable application and identifies necessary validation work, without claiming a novel detector, measured food mass, or medical efficacy.

Index Terms—food recognition, nutrition-aware recommendations, YOLO, Indian food dataset, dietary assessment, macronutrients

SUBMISSION-BLOCKING AUDIT: SEPTEMBER 2026

This is a working draft, not a submission-ready manuscript. A read-only audit of all 48,693 exported images found 138 groups of byte-identical and decoded-pixel-identical images spanning splits (63 train-validation, 56 train-test, and 19 validation-test groups), affecting 276 images. Strict annotation validation also flagged 65 images, including zero-width boxes. Historical detector metrics below are retained for traceability, but do not establish clean held-out generalization. Source exports already include augmented variants, so training-only augmentation in the merged export does not establish independence. Dataset lineage, grouped splitting and independent evaluation must be resolved before submission.

Lookup coverage was 19/72 for normalized exact matching, 55/72 without precomputed mappings and 72/72 for the current mapping layer. These are availability counts, not independently verified correctness. The historical “VERIFIED” label is a heuristic macro/mapping check, not clinical or recipe validation. Machine-readable results, fingerprints, review tem-

plates and the recovery protocol accompany this repository under `research/`.

The September application uses Next.js 16, explicit 25–1,000 g portions in 25 g steps, and browser-only history. Only explicitly saved meals contribute to daily totals. Photos and health selections are not persisted in the journal. Recommendation inputs identify database nutrition per 100 g and do not include selected grams or saved history. Health badges reflect the returned context, not medical certification. Older screenshots depict historical interface states.

A source-family recovery pass then indexed 40,797 source images and joined exact identities with within-export Roboflow filename families. It found 5,548 candidate families spanning old partitions. A deterministic review-only assignment places 26,384 images in train, 5,655 in validation and 5,652 in test, with all 72 classes represented in each. Another 11,002 images are quarantined by whole group for unresolved lineage or annotation review, not deleted. Known group overlap is zero in the candidate partitions, but upstream augmentation, within-group repeats and unrecognized cross-source near duplicates remain limitations. This is preparation for review, not a new benchmark result; the existing detector was not retrained.

I. INTRODUCTION

Image-based food recognition is often evaluated as a visual classification or detection problem. In a diet-assistance setting, however, the predicted label is only an intermediate result. A class such as `palak_paneer`, `rice`, or `samosa` must be resolved to a nutrition entry before the system can report calories, protein, carbohydrates, or fat. An incorrect resolution can produce a recommendation that appears precise while being nutritionally wrong.

The problem is pronounced in Indian food images. A single meal may contain several dishes, and the same dish may appear under regional names, spelling variants, or dataset-specific labels. Plain rice, for example, may be recorded as `WhiteRice`, `satham`, or `rice`. A chutney label may refer to a side dish or to a small portion attached to a larger item. Visually similar dishes such as `biryani` and `pulao` may also

differ in ingredient composition and macro values. These cases make direct string matching a weak basis for nutrition lookup.

This paper presents an educational implementation and an evidence audit. YOLO11s predicts visible food labels; the nutrition service selects existing database rows; the frontend scales known per-100-g values by user-estimated grams. Separately, recommendations receive included food names, per-100-g values, a goal and session-only health context. This separation permits software testing but does not demonstrate recipe equivalence or advice quality.

The contributions are:

- 1) a reproducible audit of a 72-class merged export, including exact split overlap, annotation flags and source-family recovery tooling;
- 2) a provenance-preserving lookup layer and measured coverage ablation, with independent correctness review still required;
- 3) a FastAPI and Next.js prototype that links detection, macro calculation, serving adjustment, and personalized recommendations.

II. RELATED WORK

Prior work relevant to this study can be grouped into Indian food recognition, nutrition database alignment, and diet recommendation systems. Methods for depth-based volume estimation and calibrated portion measurement are related, but they are outside the implemented scope because the present system reports database-backed per-100g macro values and leaves serving adjustment to the user.

Indian food recognition studies show that domain-specific data are necessary, but they do not fully address nutrition grounding. IndianFoodNet uses YOLO-style detection for Indian food items and is therefore closer to real meal photographs than single-label classification, where one image is forced into one category [1]. Khana provides a larger Indian cuisine benchmark and reports recurring difficulties such as class imbalance, regional variation, and visual similarity between dishes [2]. The Indian Thali study moves toward cluttered meal analysis through segmentation and controlled capture, although such capture assumptions are less convenient for ordinary web uploads [3]. Lightweight CNN and MobileNetV3 studies support compact architectures and augmentation for food recognition, but their experimental settings remain closer to classification than multi-item detection [4], [5]. These works motivate a detector trained on Indian food labels, but they leave the detector-to-nutrition link largely under-specified.

Nutrition database work addresses a different part of the problem. The Indian Food Composition Database study shows why cooked Indian foods require dedicated nutrient records rather than direct assumptions from raw ingredients [6]. The Synergistic Frameworks study identifies a practical compatibility gap: visual food labels and food-composition records often differ in spelling, language, or preparation name [7]. FKG.in demonstrates how language models and food knowledge graphs can parse regional recipe descriptions, but that setting begins with text rather than an image [8]. The present

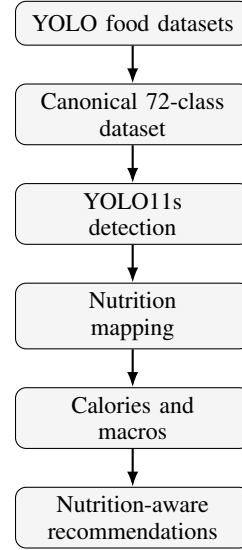


Fig. 1. End-to-end workflow for AI-based food recognition with nutrition-aware recommendations.

system therefore uses deterministic lookup, reviewed mappings, and recorded source metadata instead of delegating nutrient estimation to a language model.

Recent recommendation systems illustrate both the utility and the risk of generated diet advice. NutriGen and other web-based meal-planning systems generate personalized plans from structured user constraints [9], [10]. Diet Engine combines food detection with NLP-based dietary guidance and is close to the application direction of this work [11]. At the same time, evaluations of ChatGPT-style nutrient estimation from meal photographs show that well-formed responses do not guarantee accurate calorie or macro estimates [12]. The proposed design follows a conservative boundary: recommendations are generated after detection and macro calculation, while the numerical nutrition layer remains database-backed and inspectable.

III. PROPOSED SYSTEM

Fig. 1 summarizes the implemented system. The design uses a modular pipeline rather than a single end-to-end predictor because each stage has a different failure mode. Dataset normalization controls the detector label space, YOLO11s performs multi-item food detection, the nutrition layer resolves labels to database rows, and the recommendation module receives structured macro values. YOLO11s was selected as a compact detector suitable for a web prototype where inference time, model size, and multi-object localization are practical constraints. Recommendation text is generated only after the backend has calculated nutrition values.

Let a detector return

$$D_x = \{(\hat{c}_q, b_q, p_q)\}_{q=1}^{Q_x}, \quad (1)$$

where $\hat{c}_q$ is the predicted food class, b_q is the bounding box, and p_q is the confidence score. Let the nutrition database be

$$N = \{(n_j, \mathbf{m}_j, z_j)\}_{j=1}^L, \quad (2)$$

where n_j is a food name, $\mathbf{m}_j = [e_j, \text{protein}_j, \text{carb}_j, \text{fat}_j]$ is the nutrition vector, and z_j stores source metadata.

The following rule summarizes offline mapping construction. Runtime first uses precomputed entries, then normalized exact matching, containment with a length ratio of at least 0.5, variation containment, and last-resort fuzzy matching at 0.4. The offline 0.78 threshold does not govern every runtime fallback. These rules are preserved, not certified as clinically appropriate:

$$M(c) = \begin{cases} M_{\text{manual}}(c), & \text{if a precomputed mapping exists,} \\ M_{\text{exact}}(c), & \text{if normalized names match,} \\ M_{\text{sup}}(c), & \text{if an accepted supplemental row exists,} \\ M_{\text{fuzzy}}(c), & \text{if } \text{score}(c, n) \geq 0.78, \\ \emptyset, & \text{otherwise.} \end{cases} \quad (3)$$

The last case is a deliberate design choice. If no accepted nutrition match exists, the system keeps the detection visible but does not assign macro values. For a diet-assistance interface, an explicit missing value is preferable to a confident but unsupported nutrient estimate.

For a meal image, the displayed macro vector is

$$\hat{\mathbf{m}}_{\text{known}}(x) = \sum_{q: M(\hat{c}_q) \neq \emptyset} s_q \mathbf{m}_{M(\hat{c}_q)}, \quad (4)$$

Here $s_q = w_q/100$ for an included detection with user-estimated weight w_q in grams, and $s_q = 0$ for an excluded detection. Unknown nutrition is omitted from the known subtotal and separately marks the estimate incomplete, not zero. Portions default to 100 g and range from 25 to 1,000 g in 25 g steps. The image does not measure food mass. Only saved meals contribute to local-day totals; a successful scan alone records no consumption.

A. Runtime Data Contract

The backend response is structured so that the frontend does not infer missing or approximate values from presentation logic. Each detection carries the raw detector label, display name, confidence score, bounding box, image dimensions, macro values, nutrition unit, nutrition source, and missing-nutrition flag. If no valid food is detected, the API returns an explicit no-food state. If a detected class has no accepted nutrition match, the detection remains visible while macro values remain unavailable. The interface exposes source metadata and approximate mapping notes. Successful lookup does not establish equivalence with the photographed dish.

IV. DATASET CONSTRUCTION

Five source exports were merged into one label-normalized dataset [13], [14], [15], [16], [17]. Their names and class orders differ, so source labels were normalized for case, punctuation and selected semantic aliases before rewriting YOLO class indices. This engineering step preserves multi-food annotation rows but does not establish independent source families or resolve image-level reuse rights. The September audit qualifies all downstream detector claims.

TABLE I
SOURCE DATASET POOL BEFORE PROJECT-SPECIFIC MERGING.

Source dataset	Classes	Train	Valid	Test
Indian food detection v1	17	7547	1053	453
Indian food v6	29	8088	213	105
Indian food v2 subset	7	1698	159	92
IndianFoodNet YOLO export	30	11387	1073	576
South Indian food detection v19	31	6478	1294	581

TABLE II
REPRESENTATIVE GROUPS IN THE 72-CLASS INDIAN-FOOD LABEL SPACE.

Group	Example classes
Rice dishes	rice, lemon_rice, sambar_satham, veg_briyani
Breads	roti, paratha, bhakri, bhatura, puri
Curries	chole, rajma_curry, fish_curry, palak_paneer
Snacks	samosa, onion_pakoda, medu_vada, khandvi
Sweets	gulab_jamun, ras_malai, kheer, jalebi
Sides	coconut_chutney, green_chutney, raita, salad

TABLE III
GENERATED DATASET SPLIT SUMMARY.

Item	Result
Canonical food classes	72
Train image-label pairs	36,852
Validation image-label pairs	5,922
Archived test image-label pairs	5,919
Total exported pairs	48,693
Classes with visual samples	72 / 72
Strict-audit flagged images	65

The generated dataset used a 70/15/15 target ratio. Multi-label images were retained. Additional augmentation was restricted to training, but some source exports already contained augmented variants. The September audit found identical images spanning partitions; independence cannot be inferred from this splitting procedure.

A visual verification script sampled real images for every class and generated contact sheets. This check confirmed that all 72 classes had image samples after merging. It also found three local annotation issues: one jalebi sample that did not visually match jalebi, one kaara_chutney box placed on a vada, and one extremely thin nandu_masala box. These issues were retained as documented dataset defects rather than treated as class-order errors.

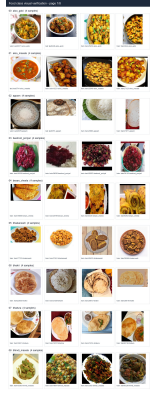


Fig. 2. Sample visual verification sheet generated from real exported dataset images.

V. HISTORICAL DETECTOR TRAINING AND EVALUATION

The final detector was trained with YOLO11s using 640-pixel images and batch size 16. Training resumed from epoch 56 and continued to epoch 100 on a Tesla T4 GPU. This configuration was chosen to keep the detector small enough for prototype inference while retaining multi-object localization, which is required for meal images containing more than one food item. The best logged validation mAP@50 was 0.83379 at epoch 78. On the archived test split, the exported model reached 0.842 precision, 0.801 recall, 0.820 mAP@50, and 0.680 mAP@50:95 across 5,919 images and 8,875 instances.

TABLE IV

HISTORICAL DETECTOR METRICS ON OVERLAPPING PARTITIONS; NOT AN INDEPENDENT BENCHMARK.

Split	Precision	Recall	mAP@50	mAP@50:95
Validation	0.847	0.807	0.830	0.692
Archived test	0.842	0.801	0.820	0.680

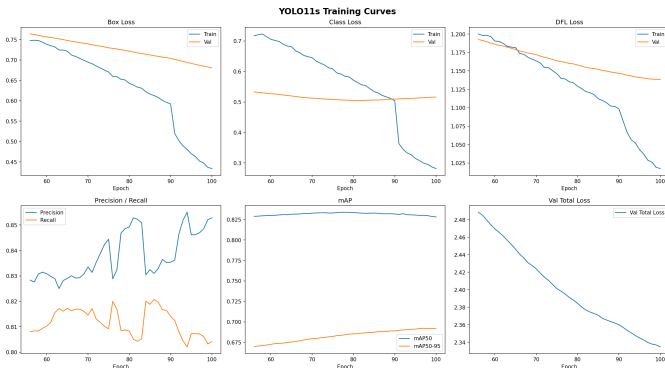


Fig. 3. YOLO11s training curves for the 72-class Indian-food detector.

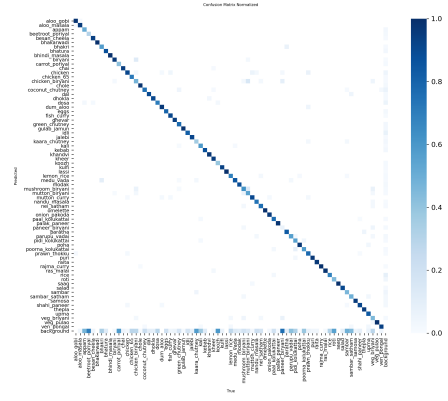


Fig. 5. Normalized confusion matrix from archived test evaluation.

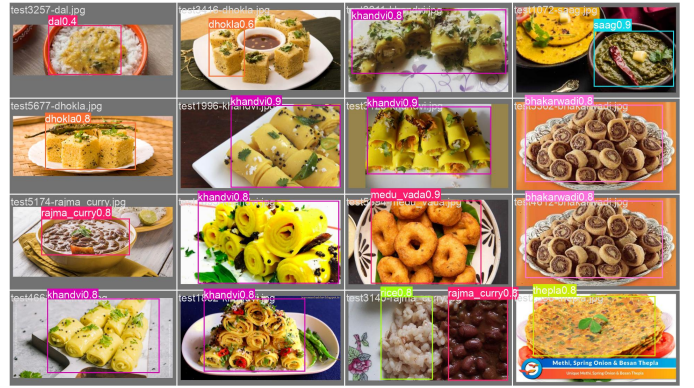


Fig. 4. Example archived test predictions from the final YOLO11s detector.

The archived validation and test metrics are numerically close, but this does not rule out leakage. The confirmed split overlap prevents treating these numbers as proof of independent generalization. Confusion matrices and qualitative predictions remain descriptive historical artifacts; source-grouped evaluation and per-class analysis are required before stronger claims.

VI. NUTRITION MAPPING AND RECOMMENDATIONS

The SQLite database imports the *Indian Nutrient Databank* workbook from the authors' public repository [6], [18]. The local workbook is byte-identical to the pinned upstream artifact; its fingerprint and source revision are recorded in `research/evidence/nutrition-provenance.json`. Artifact identity does not establish permissions for every upstream input or validate nutrient values independently. The table contains 1,010 rows, including five supplements for bhakarwadi, ghevar, jalebi, khandvi, and nandu_masala [19], [20], [21], [22], [23].

TABLE V
REPRESENTATIVE DETECTOR-TO-NUTRITION MAPPINGS.

Detector class	Nutrition database row	Score
aloo_gobi	Potato cauliflower (Aloo gobi)	1.00
palak_paneer	Spinach paneer (Palak paneer)	1.00
rice	Boiled rice (Uble chawal)	1.00
chicken_biryani	Chicken pulao	0.80
ven_pongali	Plain khitchdi	0.75

Table V records developer-selected mappings and heuristic scores. Neither a score of 1.00 nor the historical “VERIFIED” label establishes recipe equivalence. Lower scores flag approximations, while all 72 mappings require independent reference assessment. The runtime interface exposes the database source and a mapping caution.

TABLE VI
NUTRITION LOOKUP COVERAGE AND HEURISTIC CHECKS, NOT ACCURACY.

Validation item	Result
Nutrition food records	1,010
YOLO-to-database mappings	103
Expanded classes checked	72
Expanded classes with mappings	72
Heuristic score at least 0.9	51
Review mappings with complete macros	21
Rows missing required macro values	0
Expanded-class coverage	100.00%

The recommendation request contains included food names, their database nutrition per 100 g, the selected goal and health context. It does not contain target calories, portion weights, meal totals or journal history. The prompt prohibits treating its inputs as consumed portions. Production uses asynchronous Groq followed by local rule-based fallback, with a 20-second provider timeout and no automatic retries. Generated advice remains fallible and is labeled as general guidance, not medical advice.

The macro engine converts a calorie target into carbohydrate, protein, and fat grams. A selected goal defines the base macro split, and a health profile modifies that split before normalization. For a calorie target T and final macro percentages p_c , p_p , and p_f , the target grams are:

$$G_c = \frac{Tp_c}{4 \times 100}, \quad G_p = \frac{Tp_p}{4 \times 100}, \quad G_f = \frac{Tp_f}{9 \times 100}. \quad (5)$$

Macro targets are calculated through a separate endpoint and are not sent to the recommendation endpoint. The percentages and condition modifiers are existing application policy, not clinically validated targets. Returned recommendation context must match the active request before an “Adjusted for diabetes” or “Adjusted for high blood pressure” badge appears. Superseded requests are cancelled and stale advice is hidden immediately.



Fig. 6. Prototype output showing detected foods with bounding boxes and nutrition-linked results.

TABLE VII
STORED BASE WEIGHTS BEFORE NORMALIZATION AND HEALTH-CONTEXT ADJUSTMENT; NOT CLINICAL TARGETS.

Goal	Carbs	Protein	Fat
Weight loss	40%	35%	25%
Muscle gain	40%	30%	30%
Maintenance	50%	25%	25%
Endurance	55%	20%	20%

VII. PROTOTYPE VALIDATION

The deployed prototype uses FastAPI with CPU-only Torch and lazy YOLO loading, read-only SQLite lookups, and a Next.js 16 static frontend. Users follow Scan, Review portions, Save meal. They can exclude detections and edit saved portions. Daily totals use only saved records. Unknown nutrition marks subtotals incomplete. Version-1 browser storage contains meals and goal preferences, but no photos or health selections. Scanning remains usable when storage fails.

TABLE VIII
MAIN IMPLEMENTATION COMPONENTS AND RESPONSIBILITIES.

Component	Responsibility
Dataset builder	Parses source YOLO labels, normalizes class names, exports canonical splits
Vision service	Loads YOLO weights, runs inference, filters boxes, formats detections
Nutrition service	Loads SQLite rows, applies mappings, returns macros and provenance
Macro engine	Converts calorie targets and health profiles into macro goals
Recommendation service	Generates advice from detected foods and computed macros
Frontend interface	Handles upload, overlays, serving controls, macro display, and reset flow

The image-analysis response includes the detector label, display name, confidence, bounding box, image dimensions, macro values, nutrition source, and missing-nutrition flags. Runtime checks were run for classes that are easy to mis-handle, including aloo_gobi, palak_paneer, rice, AlooGobi, and WhiteRice. These labels resolved to the

expected nutrition rows. The API also returns an explicit no-food state when no valid detection is found.

TABLE IX
RUNTIME BEHAVIOR CHECKED DURING PROTOTYPE VALIDATION.

Condition	Expected behavior
aloo_gobi	Resolve to potato cauliflower nutrition row
palak_paneer	Resolve to spinach paneer nutrition row
WhiteRice	Normalize legacy label to boiled rice
No detected food	Return empty detections and no-food flag
Unmapped class	Preserve detection and mark nutrition unavailable

VIII. VALIDATION SCOPE

Validation was performed at several levels because the user-facing nutrition result depends on more than detector accuracy alone. Dataset checks confirmed class folders, label files, split counts, and visual samples. Model evaluation measured detection performance on validation and archived test splits. Nutrition checks verified that detector classes resolve to complete macro rows. Runtime checks confirmed that the backend response contains the fields required for overlays, nutrition display, serving controls, and recommendation prompts.

TABLE X
VALIDATION LAYERS USED IN THE PROJECT.

Layer	What was checked
Dataset	Split counts, class coverage, visual samples, annotation issues
Detector	Precision, recall, mAP@50, mAP@50:95, prediction examples
Nutrition	Database rows, mapping coverage, macro completeness, sources
API	Upload validation, no-food state, mapped and unmapped detections
Frontend	Overlay rendering, nutrition cards, serving controls, recommendations

The layered validation separates failure modes that would otherwise be hidden inside a single demonstration. A detector can classify a food correctly while the nutrition service maps it poorly. A nutrition database can contain complete rows while the frontend receives fields under unexpected names. Reporting these checks separately makes the system easier to audit and clarifies what has, and has not, been validated.

IX. DISCUSSION

The results show that nutrition-aware food recognition cannot be evaluated by detector mAP alone. A visually correct prediction can still produce an incorrect dietary result if it is mapped to the wrong food-composition row. The mapping table, source metadata, and explicit missing-value behavior are therefore part of the technical pipeline, not only backend implementation details.

The system also reflects a practical trade-off. It does not attempt exact portion estimation from a single image, because

that would require assumptions that are difficult to verify in ordinary user photographs. Instead, it reports database-backed macro values and lets the user adjust gram-based portions. This limits automation, but it keeps the reported nutrition values closer to the evidence available to the system.

X. LIMITATIONS

The system does not estimate food mass, verify recipe ingredients or establish medical suitability. Detection performance has not been reassessed independently after the leakage findings. All mappings, including high-scoring entries, require independent evaluation; 21 have heuristic review flags. Source-family inference cannot rule out differently named crops or recompressed copies. Upstream augmented variants and annotation conflicts need review before a benchmark is frozen. Asset rights and author approval remain unresolved.

XI. CONCLUSION

This work implements *AI-Based Food Recognition with Nutrition-Aware Recommendations* as an educational web application and audits the evidence behind its detector and nutrition layer. The existing model and 72-class export remain unchanged. Exact cross-split duplication and unresolved source-family relationships prevent interpreting the archived detector metrics as independent generalization. The mapping layer provides lookup coverage for all 72 classes, but independent recipe-equivalence assessment is outstanding. Gram-based review, saved-only daily totals, explicit source details, and response-context labels improve inspectability; software tests do not establish clinical validity. The next research milestone is reviewed source grouping and annotation handling followed by a frozen evaluation protocol, appropriate retraining or untouched external testing, and independent mapping assessment. Asset permissions and author approval remain prerequisites for submission.

ACKNOWLEDGMENT

The implementation and dataset engineering artifacts used in this manuscript were developed as part of a final year project in the Department of Computer Science and Engineering, National Institute of Technology Srinagar. The authors acknowledge the technical supervision and review provided by project guide Dr. Lavanya Madhuri Bollipo and project co-guide Dr. Insha Majeed.

REFERENCES

- [1] R. Agarwal, T. Choudhury, N. J. Ahuja, and T. Sarkar, "IndianFoodNet: Detecting indian food items using deep learning," *International Journal of Computational Methods and Experimental Measurements*, vol. 11, no. 4, pp. 221–232, 2023.
- [2] O. Prabhu, "Khana: A comprehensive Indian cuisine dataset," *arXiv preprint arXiv:2509.06006*, 2025.
- [3] Y. Arora, A. Arun, and C. V. Jawahar, "What is there in an Indian Thali?" in *Proceedings of the 16th Indian Conference on Computer Vision, Graphics and Image Processing*, 2025.
- [4] R. U. Haque, R. H. Khan, A. S. M. Shihavuddin, M. M. M. Syeed, and M. F. Uddin, "Lightweight and parameter-optimized real-time food calorie estimation from images using CNN-based approach," *Applied Sciences*, vol. 12, no. 19, p. 9733, 2022.

- [5] J. Patel and K. Modi, "Indian food image classification and recognition with transfer learning technique using MobileNetV3 and data augmentation," in *Engineering Proceedings*, vol. 56, no. 1, 2023, p. 197.
- [6] A. Vijayakumar, H. B. Dubasi, A. Awasthi, and L. M. Jaacks, "Development of an Indian food composition database," *Current Developments in Nutrition*, vol. 8, no. 7, p. 103790, 2024.
- [7] "Synergistic frameworks for Indian food computing: An analysis of nutritional and visual dataset compatibility," 2025.
- [8] S. K. Gupta, L. Dey, P. P. Das, G. Trilok-Kumar, and R. Jain, "Enhancing FKG.in: Automating Indian food composition analysis," *arXiv preprint arXiv:2412.05248*, 2024.
- [9] S. Khamesian, A. Arefeen, S. M. Carpenter, and H. Ghasemzadeh, "NutriGen: Personalized meal plan generator leveraging large language models to enhance dietary and nutritional adherence," *arXiv preprint arXiv:2502.20601*, 2025.
- [10] S. Hussain, N. Khan, S. A. A. Shah, S. Bano, and A. W. Khan, "AI-driven personalized meal planning: A web-based platform for tailored nutrition and health management," *International Journal of Computer Applications*, vol. 187, no. 57, pp. 17–29, 2025.
- [11] A. M. Saad, M. R. H. Rahi, M. M. Islam, and G. Rabbani, "Diet engine: A real-time food nutrition assistant system for personalized dietary guidance," *Food Chemistry Advances*, vol. 7, p. 100978, 2025.
- [12] C. O'Hara, G. Kent, A. C. Flynn, E. R. Gibney, and C. M. Timon, "An evaluation of ChatGPT for nutrient content estimation from meal photographs," *Nutrients*, vol. 17, no. 4, p. 607, 2025.
- [13] project-z0tql, "Indian food detection, version 1," <https://universe.roboflow.com/project-z0tql/indian-food-detection-kzw9g/dataset/1>, source/version recovered from local dataset export metadata, inspected 2026-09-12; image-level rights require separate review.
- [14] microplastics-vypxl, "Indian food, version 6," <https://universe.roboflow.com/microplastics-vypxl/indian-food-txofy/dataset/6>, source/version recovered from local dataset export metadata, inspected 2026-09-12; image-level rights require separate review.
- [15] indianfood, "Indianfood-7, version 2," https://universe.roboflow.com/indianfood/indian_food-pwzlc/dataset/2, source/version recovered from local dataset export metadata, inspected 2026-09-12; image-level rights require separate review.
- [16] IndianFoodNet contributors, "Indianfoodnet-30, version 1," <https://universe.roboflow.com/indianfoodnet/indianfoodnet/dataset/1>, source/version recovered from local dataset export metadata, inspected 2026-09-12; image-level rights require separate review.
- [17] bharani-lucid, "South indian food detection, version 19," <https://universe.roboflow.com/bharani-lucid/south-indian-food-detection/dataset/19>, source/version recovered from local dataset export metadata, inspected 2026-09-12; image-level rights require separate review.
- [18] A. Vijayakumar, H. B. Dubasi, A. Awasthi, and L. M. Jaacks, "Indian Nutrient Databank (INDB) workbook," <https://github.com/lindsayjaacks/Indian-Nutrient-Databank-INDB/blob/fbdb62bec519c6e2468582a799fb47e194b42558/INDB.xlsx>, 2024, workbook modified November 18, 2024; accessed September 12, 2026. Local file byte identity verified; see repository provenance evidence.
- [19] FatSecret India, "Evolve bhakarwadi nutrition facts per 100 g," Online nutrition record, 2026, accessed 2026-06-06.
- [20] Clearcals, "Ghevar recipe calories and nutrition per 100 g," Online nutrition record, 2026, accessed 2026-06-06.
- [21] —, "Jalebi recipe calories and nutrition per 100 g," Online nutrition record, 2026, accessed 2026-06-06.
- [22] —, "Khandvi recipe calories and nutrition per 100 g," Online nutrition record, 2026, accessed 2026-06-06.
- [23] —, "Nandu kari recipe calories and nutrition per 100 g," Online nutrition record, 2026, accessed 2026-06-06.